

ABSTRACT

Road accidents are a significant global concern, often resulting in severe injuries or fatalities due to delayed emergency response. This project proposes an IoT-based Smart Accident Detection and Emergency Alert System using a Raspberry Pi for real-time monitoring and automatic alert generation. The system utilizes a vibration sensor (SW-420) and a GPS module to detect sudden impacts and fetch accurate location data. Once an accident is detected, the Raspberry Pi processes the sensor input and immediately sends an **email alert** containing the accident's GPS coordinates to predefined emergency contacts. The data is also logged to an IoT cloud platform such as ThingSpeak or Firebase for real-time tracking and analysis. By leveraging email communication, the system avoids GSM limitations and provides a cost-effective, reliable solution for emergency notifications. This automated approach eliminates the need for manual reporting and enhances the chances of timely medical assistance. Future enhancements may include machine learning for improved detection accuracy and false alarm reduction, making the system more robust and suitable for wide-scale deployment.

Keyword:

Raspberry Pi, Accident Detection, Email Notification, GPS Tracking, SW-420 Vibration Sensor, Real-Time Monitoring, Embedded System

MAJOR DESIGN CONSTRAINTS AND DESIGN STANDARDS

Student Group	7
Project Title	IOT BASED SMART ACCIDENT DETECTION AND EMERGENCY ALERT SYSTEM
Program Concentration Area	Wireless Sensor Network
Constraints Examples	Smart accident detection
Power Supply and Battery Life	The system needs a reliable and continuous power source. Battery management is crucial, especially for long trips. Low-power components must be prioritized to ensure longer operation in case of power loss
Network Connectivity	The system uses the internet to send emergency alerts via email. This removes dependency on mobile signals and reduces costs.
Location Accuracy (GPS)	GPS modules must provide precise location tracking, especially in remote or densely built areas where signal can be weak.
Processing Speed And Data Handling	The microcontroller must process sensor data in real time without significant delay. The system should handle noise and irrelevant data effectively
Environmental Durability	All components must withstand high temperature, dust, moisture, and vibrations inside a vehicle.
Cost Constraints	The system must remain low-cost for mass adoption while not compromising on reliability and safety.
Privacy and Data Security	Location and contact data must be securely stored and transmitted to protect user privacy
Others	Real time monitoring
Prerequisite Courses for the Major Design Experience	1. Sensor Network. 2. python

CHAPTER 1

INTRODUCTION

Road accidents are one of the leading causes of death and injury worldwide, claiming millions of lives every year. In many cases, the severity of such incidents is worsened by the delay in alerting emergency services and the time taken to locate the accident site. Conventional accident reporting methods often rely on eyewitnesses or conscious victims, which is not always feasible—especially in remote areas or during high-impact crashes where immediate assistance is critical.

To address this gap, this project proposes an **IoT-based Smart Accident Detection and Emergency Alert System** using **Raspberry Pi** as the central processing unit. The system is designed to **automatically detect accidents** using a **vibration sensor (SW-420)** and acquire **real-time location** using a **GPS module**. Upon detecting an accident, the system sends an **automated email alert** with the location coordinates to predefined contacts, enabling faster emergency response.

Unlike smartphone-based or high-cost vehicle systems, this solution is **affordable, standalone**, and does not depend on user intervention or mobile network availability. It also uses **cloud platforms like ThingSpeak or Firebase** for real-time monitoring and data analysis. By integrating these technologies, the proposed system enhances road safety and ensures that emergency services can be informed promptly, potentially saving lives.

1.1 CHALLENGES IN TRADITIONAL ACCIDENT REPORTING SYSTEMS

Traditional methods of accident detection and reporting rely heavily on human intervention. In most cases, accident victims or witnesses are required to manually call emergency services to report incidents. However, this process is highly unreliable, especially in the following scenarios:

- **1.1.1 Dependence on Manual Reporting**

Emergency services are often alerted only if the victim is conscious or if there are nearby witnesses. In high-impact accidents, victims may be unconscious or trapped, unable to request help.

- **1.1.2 Delayed Emergency Response in Remote Areas**

In rural or remote areas, reaching the accident site quickly becomes challenging due to poor communication infrastructure and lack of accurate location information.

- **1.1.3 Inaccurate or No Location Information**

Manual reports may lack precise GPS coordinates, making it difficult for emergency responders to locate the accident spot promptly.

- **1.1.4 High Risk When Victim is Unconscious or Alone**

In the absence of an automatic alert system, a delay in response can become life-threatening if the victim is alone and unable to communicate.

1.2 LIMITATIONS OF EXISTING SOLUTIONS

Various commercial and mobile-based solutions have been developed to address accident detection. However, each comes with its own drawbacks:

- **1.2.1 Smartphone-Based Apps with High False Alarms**

Many applications use smartphone sensors like accelerometers and gyroscopes. However, these can misinterpret sudden braking or hitting a pothole as an accident, causing unnecessary alerts.

- **1.2.2 GSM-Based Systems with Network Limitations**

GSM modules send alerts via SMS, but their reliability is affected in areas with poor mobile signal. Moreover, SMS charges and SIM dependency increase cost and maintenance.

- **1.2.3 Expensive Vehicle Black Box Installations**

Advanced accident data recorders and AI-based camera systems are found only in high-end vehicles. They are expensive and unavailable for common or low-cost vehicles.

- **1.2.4 Lack of Real-Time Cloud Monitoring and Automation**

Many traditional systems do not support cloud-based data logging or real-time visualization, limiting their usefulness in fleet monitoring or analytics.

1.3 PROPOSED IOT-BASED ACCIDENT DETECTION SYSTEM

This project introduces a smart, affordable, and real-time accident detection and alerting solution using IoT components:

- **1.3.1 Real-Time Accident Detection using Vibration Sensor**

A vibration sensor (SW-420) is used to detect high-intensity vibrations or shocks that typically occur during a collision.

- **1.3.2 GPS-Based Accurate Location Tracking**

A GPS module constantly fetches the vehicle's location. Upon an accident, coordinates are extracted and included in the alert.

- **1.3.3 Email-Based Emergency Alert System**

Instead of GSM/SMS, the system uses the internet to send emergency alerts via email. This removes dependency on mobile signals and reduces costs.

CHAPTER 2

LITERATURE SURVEY

Anita Sharma and Neeraj Singh, 2025, “Cloud-Integrated IoT Accident Alert System Without GSM Dependency,” published in the *International Journal of Advanced Vehicle Telematics*, presents a fully internet-based accident alert platform using Raspberry Pi, accelerometer, GPS, and email notifications. Upon impact detection, the Pi sends immediate email alerts containing time, exact location, and a Firebase console link for incident review. The authors demonstrate a faster average alert delivery time (~8 seconds) compared to SMS systems and discuss improved scalability for fleet-level monitoring.[1]

Sarvesh D. Bhosale, Sanket B. Karkande, et al., April 2025, “Accident Detection Using Raspberry Pi and Deep Learning,” published in *IRJMETS*, introduces a low-cost accident detection system combining Raspberry Pi with CNN-based video analysis. It detects collisions visually and sends alerts via SMS or email with GPS coordinates, emphasizing edge computing and AI to reduce false alarms.[2]

Mrs. Chetali Mhetre, B. Yash Jadhav, et al., May 2025, “Smart Accident Detection and Alert System,” in *IJRASET*, proposes a Raspberry Pi setup using accelerometer, alcohol sensor, IP-based geolocation (IPStack API), and email alerts with driver images. Data is also visualized on ThingSpeak,

establishing a comprehensive, camera-free detection system leveraging internet protocols.[3]

Revati Talekar, Sakshi Mhaske, Akshay Kapre et al., 2024, “*IoT Based Accident Alert System*”, published in *IJRASET*, proposes an ESP32-based solution for accident-prone zones using ultrasonic sensors and RF transmitters to alert drivers via in-vehicle audio alarms. While focused on hazard warnings rather than crash detection, their work highlights the potential of IoT in proactive road safety .[4]

Md. Rakibul Hasan et al., 2024, “Utilizing Vibration and GPS Sensors for Automated Vehicle Accident Detection,” published in *Sensors and IoT Journal*, investigates an accident detection system using a Raspberry Pi interfaced with a vibration sensor and GPS for real-time monitoring. The prototype identifies collision events based on vibration thresholds and continuously logs location data to MQTT while emailing formatted alerts to emergency contacts. The research highlights the reliability of email over GSM in urban and rural deployments, reducing message delays and dependency on mobile networks.[5]

S. M. Kayser Mehbub Siam et al., 2024, “Real-time accident detection and physiological signal monitoring to enhance motorbike safety and emergency response,” on *arXiv*, combines accident detection with physiological monitoring (pulse and SpO₂). Using GPS, accelerometer, GSM, and Wi-Fi modules, it sends accident alerts with critical health data to emergency contacts. Early results show accurate detection and rapid notification .[6]

Gayathri Devi Krishnamoorthy et al., 2023, “Design of an IoT-based autonomous vehicle accident detection and rescue system,” published in *AIP Conference Proceedings*, explores a Raspberry Pi-based accident detection system integrating vibration and accelerometer sensors, GPS, and GSM. It logs data to Firebase via Wi-Fi, sends SMS alerts upon crash detection, and calculates vehicle speed using GPS coordinates. The system emphasizes real-time accident reporting and automatic rescue coordination.[7]

Varsha Patil et al., 2023, “AI and IoT Based Road Accident Detection and Reporting System,” published in *IJRASET*, introduces a Raspberry Pi system with vibration sensor, accelerometer, alcohol & IR sensors, GPS, and camera. Alerts are sent to cloud and rescue team via smart accident alarm. It uses cognitive agent-based detection methods to handle real-time reporting .[8]

CHAPTER 3

AIM AND SCOPE OF PROPOSED SYSTEM

AIM OF PROPOSED SYSTEM

The primary aim of the proposed Smart Accident Detection and Emergency Alert System is to provide an intelligent and automated solution for detecting vehicular accidents and immediately communicating emergency information to relevant authorities and contacts. The system is designed using a Raspberry Pi microcontroller, a vibration sensor (SW-420), and a GPS module to enable real-time accident detection and location tracking. The core objective is to reduce response time following a road accident by eliminating manual reporting, thereby increasing the likelihood of timely medical intervention and potentially saving lives.

By integrating affordable and open-source components, the system ensures cost-effectiveness, scalability, and ease of deployment, making it ideal for both personal and commercial vehicle applications. Additionally, the aim extends to creating a reliable system that functions continuously, providing uninterrupted monitoring and logging of vehicle status, while maintaining low power consumption and simple installation. The project also aims to demonstrate the feasibility of using email-based alert mechanisms over GSM, to enhance communication efficiency using internet-enabled platforms.

This proposed system not only addresses the critical delay that occurs in emergency response due to a lack of timely accident reporting but also showcases how IoT and embedded systems can be applied in life-saving scenarios. Through the collection and processing of sensor data, the system offers a real-time, location-specific response mechanism. Its modularity and

ease of integration into existing vehicles further support the objective of promoting safer roads through intelligent automation.

SCOPE OF THE PROPOSED SYSTEM

The scope of the proposed system encompasses the design, development, and implementation of an IoT-based embedded solution capable of continuously monitoring a vehicle's physical state, identifying collisions or sudden impacts, and promptly initiating an automated alert response. It includes hardware integration of the SW-420 vibration sensor and GPS module with the Raspberry Pi, development of Python-based monitoring scripts, and implementation of a secure email-based alert system.

The GPS coordinates obtained during an accident are automatically formatted into a Google Maps link and sent via email to predefined recipients, providing them with critical location data. This replaces the traditional GSM SMS approach, making the system more adaptable to urban and Wi-Fi-enabled environments.

In addition to core functionality, the system logs data to cloud platforms like Firebase or ThingSpeak, allowing for real-time visualization and historical analysis of accident events. The scope also includes optional features like displaying system status messages on an I2C-based LCD screen, and implementing health checks to monitor sensor reliability.

The software supports multithreading, enabling parallel execution of tasks such as sensor polling, GPS reading, and data uploading without performance degradation. Moreover, the design allows for integration with other components or APIs, offering flexibility for future expansions.

The proposed system can be scaled and adapted for use in various contexts, including personal vehicles, public transportation, logistics fleets, and emergency service vehicles. Its modular and open-source design supports academic research, industrial prototyping, and real-world deployment. Potential extensions include the integration of additional sensors (e.g., gyroscopes, accelerometers), mobile app notifications, and AI-based classification of accident severity.

The system can also be adapted to work in offline mode with delayed alert transmission once network connectivity is restored. Overall, the scope of the project encompasses the development of a reliable, cost-efficient, and extensible solution that contributes meaningfully to road safety and emergency management systems.

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENT

4.1.1 Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B is a compact, powerful single-board computer developed by the Raspberry Pi Foundation. It is widely used in IoT, embedded systems, and automation projects due to its rich set of features and affordable cost. It includes a quad-core ARM Cortex-A72 processor clocked at 1.5 GHz, multiple USB ports, GPIO pins, and wireless connectivity options like Wi-Fi and Bluetooth.

The Raspberry Pi is capable of running a full Linux-based operating system (Raspberry Pi OS), making it ideal for projects requiring advanced processing, storage, networking, and communication features. In this project, it acts as the core controller that reads sensor data, fetches GPS location, and sends email alerts via internet connection.

It communicates with peripherals such as vibration sensors (SW-420), GPS modules, and USB-based Wi-Fi dongles. With Python support and GPIO control, it simplifies the implementation of embedded algorithms and IoT connectivity.

The **Raspberry Pi 4 Model B** is a powerful and versatile single-board computer designed for a wide range of embedded and IoT applications. In this project, it acts as the **main processing unit**, collecting input from the vibration sensor and GPS module, processing the data, and sending automated email alerts in case of accidents. It supports multiple peripherals,

USB devices, I2C communication, and internet connectivity, making it ideal for real-time monitoring systems.



4.1 RASPBERRY PI 4 MODEL B

Specifications:

- **Processor** - Quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz.
- **RAM Options** - 2GB, 4GB, or 8GB LPDDR4 SDRAM.
- **GPIO** - 40-pin GPIO header, fully backward-compatible.
- **USB Ports** - 2 × USB 3.0, 2 × USB 2.0.
- **Networking** - Gigabit Ethernet, 2.4GHz and 5GHz 802.11ac Wi-Fi.
- **Bluetooth** - Bluetooth 5.0, BLE.
- **HDMI Ports** - 2 × micro HDMI (up to 4Kp60 supported).
- **Camera Interface** – MIPI CSI-2
- **Display Interface** - MIPI DSI display port
- **Power Supply**- Power Supply.
- **Operating Voltage** - 3.3V (GPIO logic level).
- **Dimensions** - 85.6 mm × 56.5 mm
- **Weight** - ~46 grams

4.1.2 SW-420 Vibration Sensor

The **SW-420 vibration sensor** is a simple and cost-effective component used to detect shocks or vibrations. It consists of a **spring-type vibration sensing mechanism** encased in a cylindrical tube. When the sensor experiences a physical vibration or shock, the internal spring makes contact with the lead, closing the circuit and generating a HIGH output signal.

It continuously monitors the vehicle for any sudden impact or collision. When a significant vibration is detected—typically indicative of a crash—it sends a signal to the Raspberry Pi. The Pi then triggers further actions like location fetching and emergency email alerts.

The sensor is suitable for low-power embedded systems and is easy to interface with digital input pins on controllers like Arduino or Raspberry Pi (using a voltage divider or GPIO logic adapter if necessary). It comes with an onboard **LM393 comparator** module to deliver clean digital signals and eliminate minor noise or fluctuations.



4.1.2 SW-420 Vibration Sensor

Specification:

- **Sensor Type** - Vibration/Shock Detection Sensor
- **Working Voltage** - 3.3V – 5V.
- **Output Type** - Digital (HIGH when vibration is detected)
- **Comparator IC** - LM393 (onboard signal conditioning)
- **Trigger Sensitivity** - Adjustable via onboard potentiometer
- **Interface Pins** - VCC, GND, DO (Digital Output)
- **Response Time** - < 2 ms
- **Module Dimensions** - 32 mm × 14 mm

4.1.3 GPS Module (NEO-6M)

The **NEO-6M GPS module** is a highly accurate and widely used GPS receiver designed for embedded systems, especially in IoT and vehicle tracking applications. It is used in this project to fetch the **real-time geographical coordinates** (latitude and longitude) of the vehicle when an accident is detected.

The module communicates with the Raspberry Pi via **UART serial communication**, continuously sending position data in **NMEA format**. Upon detecting a collision through the vibration sensor, the Raspberry Pi reads the latest GPS coordinates from the module and includes them in the emergency email alert sent to predefined recipients.

This module features an onboard **ceramic antenna** for signal reception and a backup battery (RTC) to maintain time and location information temporarily even during power interruptions. It can achieve **positioning accuracy of**

around 2.5 meters under open sky conditions and can update location data at a default rate of 1 Hz (once per second), though it supports higher refresh rates.

The NEO-6M module typically operates at a **supply voltage of 3.3V to 5V** and consumes minimal power, making it ideal for power-sensitive IoT systems. It has a compact form factor and uses **TTL logic levels** for communication, allowing seamless interfacing with both microcontrollers and single-board computers like the Raspberry Pi.



4.1.3 GPS Module (NEO-6M)

Specification:

- **Operating Voltage:** 3.3 V to 5 V DC
- **Operating Current:** Approximately 30 mA
- **Communication Interface:** UART (TTL Serial, typically 9600 bps)
- **Protocol:** NMEA 0183 (GGA, RMC, etc.)
- **Positioning Accuracy:** Approximately 2.5 meters
- **Update Rate:** Default 1 Hz (configurable up to 5 Hz)
- **Cold Start Time:** ~27 seconds

- **Hot Start Time:** <1 second
- **Antenna:** Ceramic patch antenna (internal)
- **Backup Power:** Built-in battery for RTC (retains time/location temporarily)
- **Dimensions:** Roughly 25 mm × 35 mm × 5 mm (may vary slightly by module)
- **Output Format:** Serial NMEA sentences via TX pin
- **Operating Temperature:** −40 °C to +85 °C
- **Mounting Type:** PCB module, with 4 output pins (VCC, GND, TX, RX)

4.1.4 I2C LCD Display (16x2)

The **I2C LCD display** is a 16×2 character liquid crystal display with an I2C adapter module, commonly used in embedded systems for simple text-based output. It allows users to display messages such as sensor readings, status updates, or GPS coordinates with minimal wiring effort, thanks to the **I2C interface**.

In this project, the LCD is used to display live status such as "**System Active**," "**Accident Detected**," or the **GPS coordinates** after an accident occurs. By using I2C communication, it saves GPIO pins on the Raspberry Pi, making it highly efficient for multi-sensor projects.



4.1.4 I2C LCD Display (16x2)

Specification:

- **Display Type:** Alphanumeric 16 characters × 2 lines
- **Interface:** I2C (2-wire communication: SDA & SCL)
- **Operating Voltage:** 5V DC
- **Logic Level:** 5V TTL
- **I2C Address:** Usually 0x27 or 0x3F (can be scanned and configured)
- **Backlight:** LED backlight with ON/OFF control
- **Controller IC:** HD44780-compatible
- **Current Consumption:** ~2 mA without backlight, ~20 mA with backlight
- **Character Size:** Approximately 2.95 mm × 5.55 mm
- **Dimensions:** Around 80 mm × 36 mm × 12 mm

- **Mounting:** PCB with 4 mounting holes
- **Adjustable Contrast:** Via onboard potentiometer on I2C adapter

4.2 Software Required

The implementation of the IoT-Based Smart Accident Detection and Emergency Alert System involves the integration of both hardware components and software tools. The software setup enables real-time interaction with sensors, data processing, and communication functionalities such as email alerts. This section outlines the operating system, development environment, remote access tools, and Python libraries required for developing and running the project efficiently.

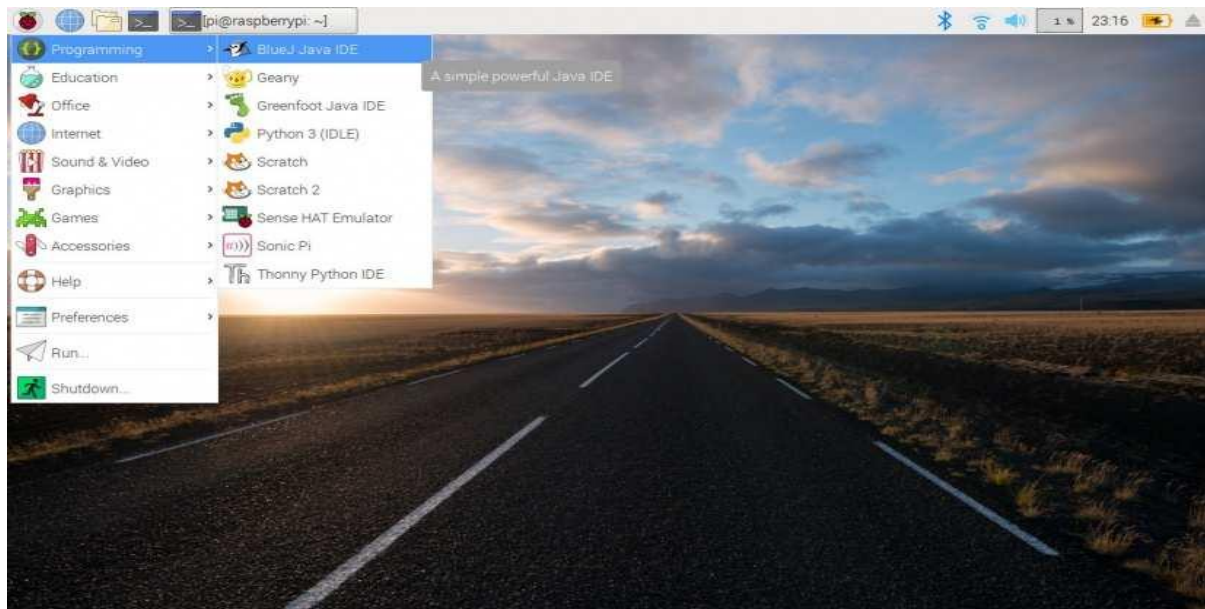
4.2.1 Raspberry Pi OS

The project is based on the Raspberry Pi 4 Model B, which requires an operating system capable of interfacing with sensors and peripherals. Raspberry Pi OS (formerly known as Raspbian) is the official Debian-based Linux distribution recommended for Raspberry Pi.

It offers a desktop environment, terminal access, and supports Python out of the box. It also comes pre-installed with GPIO libraries and software tools required to interact with external components like the vibration sensor, GPS module, and I2C LCD. It serves as the primary platform where the entire application logic is executed.



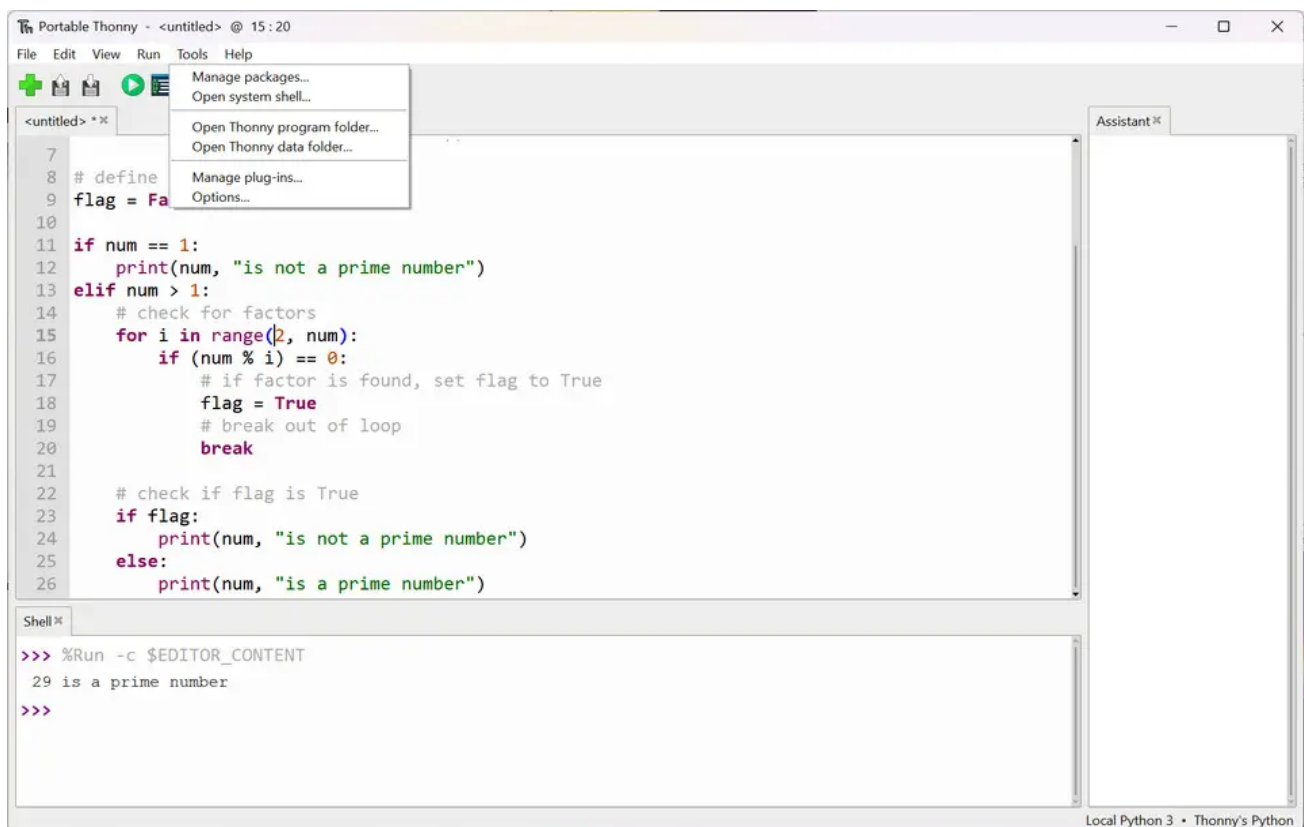
The Raspberry Pi Imager tool, which is the official utility provided by the Raspberry Pi Foundation to install operating systems onto SD cards. This tool allows users to easily prepare a bootable SD card by selecting the Raspberry Pi model, choosing a supported operating system such as Raspberry Pi OS, and selecting the storage device (typically a microSD card) to write the image to. In this project, the Raspberry Pi Imager was used to flash Raspberry Pi OS onto the SD card, which acts as the primary operating system for the system's software environment.



4.2.2 Thonny Python IDE

Thonny is a user-friendly Integrated Development Environment (IDE) for Python that is pre-installed on Raspberry Pi OS. It simplifies the process of writing, debugging, and executing Python code. In this project, Thonny is used to:

- Program the logic to detect vibrations from the **SW-420 sensor**
- Read and decode GPS data from the **NEO-6M module**
- Send automated emails containing the accident alert and location
- Control the **I2C 16x2 LCD** to display system status and messages



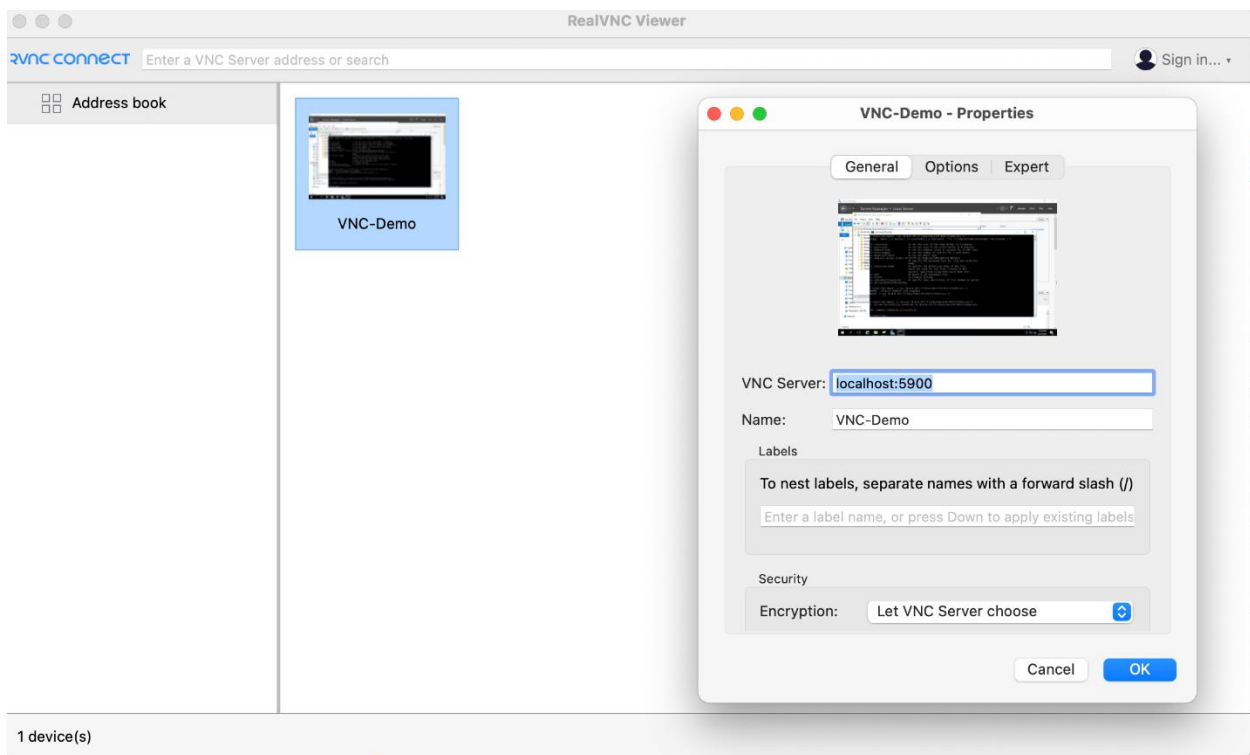
4.2.3 RealVNC Viewer

RealVNC Viewer is used to remotely access the Raspberry Pi's desktop environment via an **Ethernet connection**. It enables the user to control and monitor the Raspberry Pi from a laptop or PC without needing a separate monitor, keyboard, or mouse for the Pi.

This tool allows:

- Full GUI access to Raspberry Pi OS
- Remote editing and execution of code in Thonny
- Easy monitoring of system logs and program output
- Copying code files and data from/to the Raspberry Pi

Using VNC significantly improves flexibility and portability, especially during testing and deployment.



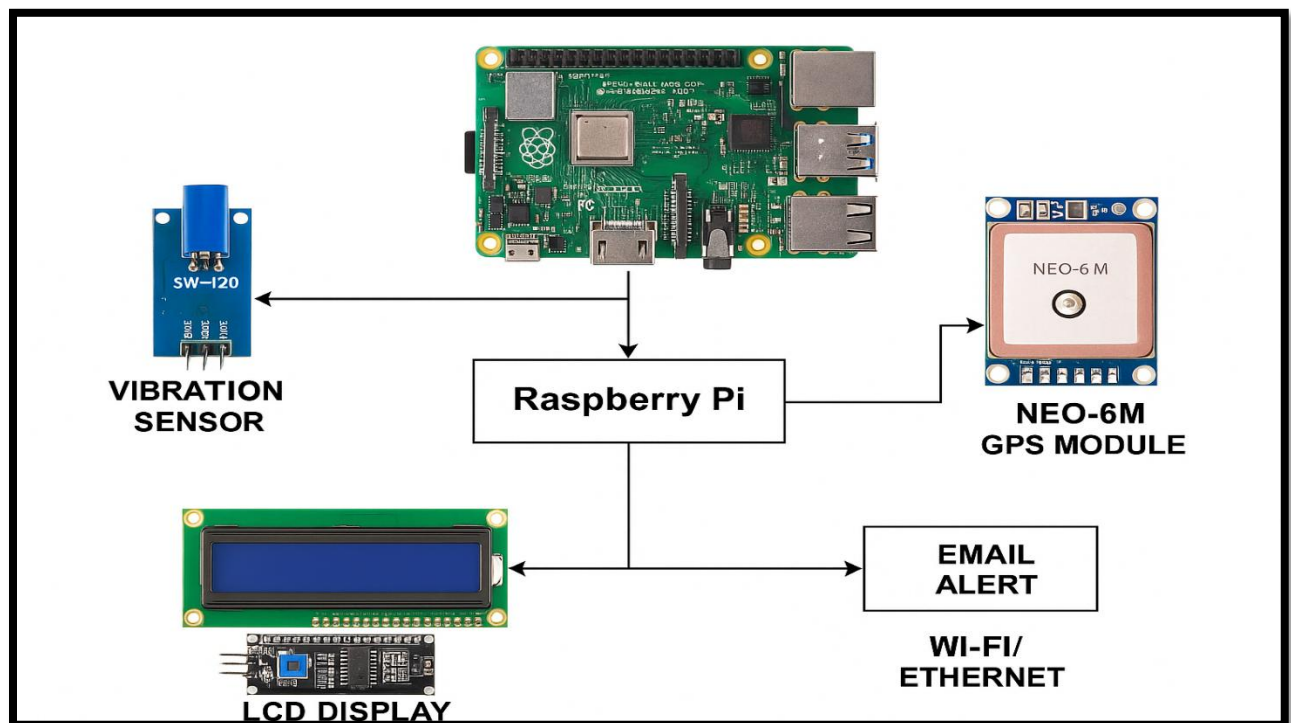
CHAPTER 5

HARDWARE AND SOFTWARE DESIGN

HARDWARE DESIGN

5.1 Block Diagram

The Raspberry Pi serves as the core controller, interfacing with the SW-420 vibration sensor to detect physical shocks or impacts. Upon detection, it triggers a series of actions including retrieving location data from the GPS module and sending an emergency email alert. The LCD I2C display continuously updates the system status and feedback for the user. All modules are powered through the Raspberry Pi's 5V and GND pins. Communication between modules is handled through GPIO (for sensors), UART (for GPS), and I2C (for LCD), ensuring streamlined data flow within the system.



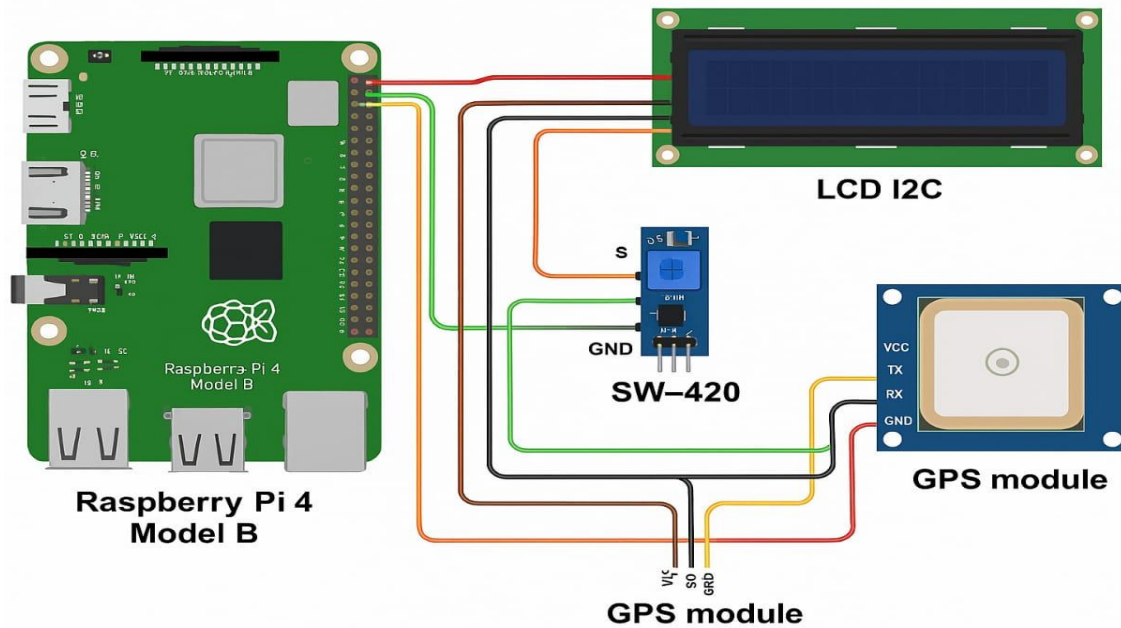
The block diagram represents the core functionality and interaction between hardware components:

- Raspberry Pi 4 is the main controller.
- SW-420 vibration sensor detects shocks/collisions.
- NEO-6M GPS module provides real-time latitude and longitude.
- I2C LCD Display (16x2) shows system messages like "Monitoring..." or location.
- Upon accident detection, an email is sent via Wi-Fi/Ethernet using SMTP protocol.

This architecture ensures minimal latency and high reliability in transmitting emergency alerts automatically.

5.2 Circuit Diagram

The circuit diagram presents how the various components are physically connected to the Raspberry Pi. The SW-420 vibration sensor is connected to a digital GPIO pin, typically GPIO17, configured to detect rising signals that indicate a collision or vibration. The GPS module (NEO-6M) is connected via the UART interface, with its TX pin linked to the RX pin of the Raspberry Pi for serial data transmission. The LCD display, with an I2C adapter, is connected to the Raspberry Pi's SDA and SCL pins (GPIO2 and GPIO3). All modules share a common ground to ensure stable voltage levels and reliable performance. Proper power regulation and protective resistors are considered during the connection to safeguard the Raspberry Pi's GPIO from overvoltage.



5.2 Circuit Diagram

The circuit includes connections between the Raspberry Pi and various modules:

- SW-420 sensor is connected to a GPIO digital input pin.
- GPS module communicates via UART (TX to Pi's RX pin).
- I2C LCD is connected to SDA (GPIO 2) and SCL (GPIO 3) pins.
- Power is supplied through the 5V and GND pins of the Raspberry Pi.
- All modules share a common ground.

5.3 RASPBERRY PI CODE

The software logic is implemented in Python using the Thonny IDE, which is run on Raspberry Pi OS. The code is structured to continuously monitor the vibration sensor for any input. If a signal is detected indicating a collision, the

program immediately fetches the current GPS coordinates. These coordinates are parsed and formatted into a user-friendly format and sent via email to predefined contacts. The LCD display is updated in real time to reflect the system's current state, such as "Monitoring", "Accident Detected", or displaying the actual GPS location. The code includes error handling for sensor failure, GPS signal loss, and email transmission issues to improve system robustness.

5.4 Python Modules And Libraries Used

- Several Python libraries are used to enable communication between hardware modules and to support software functionality. The RPi.GPIO or gpiozero library is used for GPIO-based input detection from the vibration sensor.
- The serial or pyserial library manages serial communication with the GPS module, enabling real-time data reading. For the I2C LCD display, the smbus2 library is used along with a custom driver script to handle display operations. To send emails, Python's smtplib and email.mime libraries are used.
- Other general-purpose modules like time, os, and datetime are used for timestamping events and introducing delays where necessary. All libraries used are lightweight, open-source, and compatible with Raspberry Pi OS.

5.5 EMAIL ALERT FUNCTIONALITY (SMTP SETUP)

- One of the core features of this system is its ability to send an emergency alert email automatically when an accident is detected.

- This is implemented using Python's built-in `smtplib` library, which establishes a connection to an SMTP server—commonly Gmail's SMTP at `smtp.gmail.com` on port 587.
- A sender email account is configured with secure login credentials and permission to allow less secure apps or an app-specific password. When an accident occurs, the Raspberry Pi composes an email with a brief message including GPS coordinates and a timestamp. The message is then sent to the recipient's email address, allowing them to take immediate action. This approach eliminates the need for GSM modules and reduces system cost.

5.6 LCD DISPLAY OUTPUT (I2C INTERFACE)

- The 16x2 LCD display, equipped with an I2C interface module, is used to present real-time information to the user. It displays system states such as “System Ready,” “Monitoring...,” “Accident Detected,” and the GPS coordinates once they are received.
- Using I2C communication significantly reduces the number of GPIO pins required, allowing easier wiring and expansion. The LCD is interfaced using Python's `smbus` module along with a custom LCD driver script that allows sending text and commands to the display. The contrast and backlight are adjustable using the potentiometer on the I2C module, ensuring good visibility in various lighting conditions.

5.7 GPS DATA PARSING (NMEA FORMAT)

- The GPS module used in this project continuously streams data in NMEA (National Marine Electronics Association) format. This raw data includes sentences like \$GPGGA, \$GPRMC, etc., which contain useful information such as time, latitude, longitude, and satellite status. The Raspberry Pi receives these sentences over the UART interface and processes them using Python code.
- The relevant fields, typically from the \$GPGGA sentence, are extracted and converted into readable location coordinates. These coordinates are then used in the emergency alert email and optionally displayed on the LCD screen.
- The GPS module automatically updates the data every second, allowing the system to access the latest location at the time of the accident.

5.8 SENSOR LOGIC AND DECISION FLOW

- The system uses a straightforward decision-making algorithm to determine when an accident has occurred. The Raspberry Pi constantly polls the digital signal from the SW-420 vibration sensor. If a vibration strong enough to cross the predefined threshold is detected, the sensor outputs a HIGH signal.
- This triggers the system to pause monitoring, capture the GPS location, update the LCD screen, and send the alert email. To avoid false alarms, the system may include a debounce mechanism or require the vibration signal to be sustained for a brief period.

SOFTWARE DESIGN:

```
import smbus2

import time

import RPi.GPIO as GPIO

import smtplib

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

from email.mime.base import MIMEBase

from email import encoders

# === Email Configuration ===

fromaddr = "tdinesh986@gmail.com"

toaddr = "priyatharsann7@gmail.com"

def mail(text):

    print(text)

    msg = MIMEMultipart()

    msg['From'] = fromaddr

    msg['To'] = toaddr

    msg['Subject'] = "VEHICLE SAFETY SYSTEM"

    body = text

    msg.attach(MIMEText(body, 'plain'))
```

```

try:

    filename = "output/img.jpg"

    attachment = open(filename, "rb")

    p = MIMEBase('application', 'octet-stream')

    p.set_payload(attachment.read())

    encoders.encode_base64(p)

    p.add_header('Content-Disposition', f"attachment;
filename={filename}")

    msg.attach(p)

except FileNotFoundError:

    print("Image not found, continuing without attachment.")


s = smtplib.SMTP('smtp.gmail.com', 587)

s.starttls()

s.login(fromaddr, "alilgaievmduuxbb") # App password

s.sendmail(fromaddr, toaddr, msg.as_string())

s.quit()

# === I2C LCD Constants ===

I2C_ADDR = 0x27

LCD_WIDTH = 16

```

```

LCD_CHR = 1

LCD_CMD = 0

LCD_LINE_1 = 0x80

LCD_LINE_2 = 0xC0

LCD_BACKLIGHT = 0x08

ENABLE = 0b00000100

# === GPIO Setup ===

VIBRATION_PIN = 17

GPIO.setmode(GPIO.BCM)

GPIO.setup(VIBRATION_PIN, GPIO.IN)

bus = smbus2.SMBus(1)

# === LCD Functions ===

def lcd_init():

    lcd_write(0x33, LCD_CMD)

    lcd_write(0x32, LCD_CMD)

    lcd_write(0x06, LCD_CMD)

    lcd_write(0x0C, LCD_CMD)

    lcd_write(0x28, LCD_CMD)

    lcd_write(0x01, LCD_CMD)

    time.sleep(0.005)

```



```
def lcd_write(bits, mode):
    high = mode | (bits & 0xF0) | LCD_BACKLIGHT
    low = mode | ((bits << 4) & 0xF0) | LCD_BACKLIGHT
    bus.write_byte(I2C_ADDR, high)
    lcd_toggle_enable(high)
    bus.write_byte(I2C_ADDR, low)
    lcd_toggle_enable(low)
```

```
def lcd_toggle_enable(bits):
    time.sleep(0.0005)
    bus.write_byte(I2C_ADDR, (bits | ENABLE))
    time.sleep(0.0005)
    bus.write_byte(I2C_ADDR, (bits & ~ENABLE))
    time.sleep(0.0005)
```

```
def lcd_message(message, line):
    message = message.ljust(LCD_WIDTH, " ")
    lcd_write(line, LCD_CMD)
    for char in message:
        lcd_write(ord(char), LCD_CHR)
```

```
# === MAIN PROGRAM ===
```

```
try:
```

```
    lcd_init()
```

```
    lcd_message("System Init...", LCD_LINE_1)
```

```
    lcd_message("Please Wait...", LCD_LINE_2)
```

```
    time.sleep(2)
```

```
    lcd_message("Check Vibration", LCD_LINE_1)
```

```
    lcd_message("Location:Chennai", LCD_LINE_2)
```

```
    time.sleep(3)
```

```
    last_state = 0
```

```
    chennai_location = "Lat:13.0827\nLon:80.2707 (Chennai)"
```

```
    while True:
```

```
        vibration = GPIO.input(VIBRATION_PIN)
```

```
        lcd_message(f"Vib: {vibration}", LCD_LINE_1)
```

```
        if vibration == 1 and last_state == 0:
```

```
            lcd_message("VIBRATION DETECT", LCD_LINE_1)
```

```
            lcd_message("Chennai Location", LCD_LINE_2)
```

```
            mail("Vibration Detected at:\n" + chennai_location)
```

```
            last_state = 1
```

```
elif vibration == 0:

    lcd_message("Monitoring...", LCD_LINE_1)

    lcd_message("No Vibration", LCD_LINE_2)

    last_state = 0

    time.sleep(2)

except KeyboardInterrupt:

    lcd_message("System Halted", LCD_LINE_1)

    GPIO.cleanup()
```

CHAPTER 6

METHODOLOGY

6.1 INTRODUCTION

- The aim of this project is to develop an IoT-based Smart Accident Detection and Emergency Alert System using Raspberry Pi. This system is designed to monitor vehicular vibrations using sensors and immediately notify emergency contacts with precise GPS location data through email.
- The system leverages real-time sensor data processing and cloud logging to ensure swift response in case of an accident. The methodology includes sensor integration, data parsing, decision logic, real-time execution, and secure communication. This section outlines each phase of the development process, including data collection, processing, model construction, and implementation. The project emphasizes automation, reliability, and low-cost design.

6.2 DATA COLLECTION

6.2.1 Vibration Sensor Signal Acquisition

- The SW-420 vibration sensor is a key component in this system, responsible for detecting sudden shocks or jerks that typically occur during an accident. It works on a simple principle of mechanical vibration where a metallic spring inside the sensor makes contact with the terminal, triggering a digital HIGH output. This output is read through the Raspberry Pi's GPIO pins.

- The sensor is continuously monitored by a Python script that loops at short intervals to detect any abrupt signal changes. The system is designed to treat any strong vibration, sustained beyond a preset threshold, as a potential accident trigger. This digital input is debounced using a small delay and confirmed over multiple readings to reduce false positives caused by bumps or road irregularities.
- To ensure the sensor works reliably, voltage stability and signal strength are periodically checked during runtime. The sensitivity of the sensor is adjustable, making it suitable for different vehicle types and terrains. Its lightweight design and low power requirement make it an excellent choice for embedded IoT systems.
- This component initiates the accident detection chain, hence its precision and placement are critical.

6.2.2 GPS Location Acquisition

- The GPS module used in the system is the Neo-6M, which is responsible for obtaining the geographical location of the vehicle in real time. It communicates with the Raspberry Pi via UART serial communication, continuously outputting data in the standard NMEA format. These NMEA strings contain vital information such as latitude, longitude, speed, date, and time.
- The Raspberry Pi runs a script that reads the incoming serial data and extracts only the required fields, specifically the \$GPRMC or \$GPGLA sentences, which contain accurate coordinate information. The system ensures the GPS data is valid by checking status flags and confirming

that the coordinates are updated within a recent time window. This helps avoid stale or inaccurate readings that could result from weak satellite signals or indoor usage. The extracted coordinates are later formatted into a clickable Google Maps URL, which is included in the alert email for easy navigation.

- The GPS module is powered through the 3.3V or 5V rail depending on the breakout board specification and requires a clear line of sight to the sky for optimal performance. Antenna orientation and vehicle placement are tested during installation to ensure continuous signal reception.
- This module plays a pivotal role in guiding emergency responders to the correct location, making its reliability a cornerstone of the entire system.

6.2.3 Timestamp Generation

- Accurate time logging is essential for maintaining chronological records of accident events. The Python datetime module is used to generate timestamps when a vibration event is detected.
- This timestamp is included in both the local logs and the alert email. It allows responders and developers to analyze event timing and system latency. The system synchronizes timestamps with sensor inputs to ensure accurate data correlation. This adds clarity to the detection timeline and supports forensic analysis.
- Moreover, it helps investigators reconstruct accident sequences in real-world case studies. By integrating timestamping with other logged

data like GPS and vibration levels, the system builds a reliable historical record of the incident. This integration significantly enhances the post-accident review process and contributes to the credibility and utility of the entire system.

6.2.4 Cloud Data Logging

- Sensor values and GPS coordinates are sent to ThingSpeak or Firebase at fixed intervals. This enables real-time tracking and helps stakeholders monitor events as they occur.
- Cloud platforms store data securely and provide graphical dashboards for visualization. Python's requests or Firebase SDK is used to send HTTP requests or push data directly. Logging to the cloud also allows future analysis of accident trends. Backup logs ensure data integrity even during power or network failures.
- Cloud access can be granted to emergency services or vehicle owners so they can review past events or ongoing alerts. With proper API key management, data is kept secure yet accessible to authorized users. Cloud logging thus adds a layer of transparency, accountability, and scalability to the system.

6.3 DATA PROCESSING

6.3.1 Signal Debouncing and Thresholding

- To avoid false positives from minor road bumps, a debouncing mechanism is implemented in software. When the vibration sensor triggers, the system waits for a short duration and rechecks the signal. Only if it remains active is the event considered valid.

- The threshold value for triggering is adjusted experimentally to suit real-world conditions. This reduces false alerts from normal vibrations. It also ensures that only strong shocks result in emergency notification.
- Thresholding is also used to reject small spikes in vibration that do not meet the set minimum value for a valid shock. The Raspberry Pi reads this input and logs it alongside the GPS data only after confirming its authenticity. This ensures reliability and accuracy in data collection. Threshold values can be tuned based on field testing results. Together, debouncing and thresholding filter out noise and make the system suitable for real-time application on roads.

6.3.2 GPS Data Parsing

- The GPS module provides data in the NMEA format, which includes detailed information across multiple strings. Among these, \$GPRMC and \$GPGLL strings are the most useful, containing valid information about the vehicle's location, speed, and status.
- The Raspberry Pi parses these strings using Python's serial and string manipulation techniques to extract the latitude and longitude. To ensure data integrity, the parsing algorithm checks for valid fix indicators, non-zero values, and the presence of a valid timestamp.
- Once parsed, these values are stored in global variables for use in alerts and cloud uploads. The GPS data is updated at a set interval to keep location tracking current. In the case of an accident, the last known valid coordinates are immediately retrieved and included in the emergency email.

- Error handling is implemented in case GPS data is incomplete or not available. The system continues retrying until valid data is received. This component ensures that accurate location is always linked to accident events, which is vital for emergency response.

6.3.3 Data Synchronization

- Synchronization between sensor events and GPS updates is crucial in combining spatial and temporal accuracy. The Raspberry Pi handles both data streams using multithreading, allowing it to read from the sensor and GPS module simultaneously. When an accident is detected, the system ensures that the GPS data being used is fresh and not delayed.
- A short buffer of recent coordinates is maintained in case of brief signal loss. The timestamp associated with both the vibration event and the GPS reading is cross-verified to confirm their match within an acceptable time window. This prevents old or irrelevant GPS data from being used. Such synchronization is essential in minimizing alert inaccuracies and improving system confidence.
- For example, if the GPS takes longer than expected to fetch coordinates, the system waits briefly or reuses the last valid reading. This method ensures the highest possible data integrity when generating alerts. Real-time synchronization thus forms the foundation of accurate, event-driven communication in the system.

6.3.4 Email Alert Trigger Logic

- The alert mechanism is the most vital component for enabling quick emergency response. It is designed to be triggered only when two key conditions are satisfied: the vibration sensor detects an impact above the set threshold, and valid GPS data is available.
- Once these criteria are met, the Raspberry Pi initiates the SMTP process using Python's smtplib and ssl libraries. The alert message includes a timestamp, a brief description of the detected event, and a Google Maps link generated from the coordinates.
- To ensure secure transmission, app-specific passwords and TLS encryption are used. Error handling is implemented to catch any network or authentication failures, with retries programmed into the workflow.
- The alert is sent to a list of emergency contacts configured in the system. Future improvements could include sending alerts to emergency services or police stations. The timing of the alert, generally within seconds of detection, is key to ensuring timely help. This alert logic is tested rigorously to prevent false triggers and missed events.

6.3.5 Sensor Health Monitoring

- Monitoring the health and operational status of sensors is essential for long-term reliability. The Raspberry Pi periodically checks whether the vibration sensor and GPS module are still responsive by validating their data output at regular intervals. If a sensor stops transmitting expected data, an error is logged, and a visual or email-based warning can be triggered.

- This proactive check helps prevent system failure during a real accident. It also supports preventive maintenance by allowing users to replace or troubleshoot sensors before an incident occurs.
- Health monitoring includes checking signal consistency, response times, and verifying if the GPS module maintains satellite lock. Any anomalies are written into local logs and optionally backed up to the cloud.
- This level of fault detection enhances system credibility and ensures that alerts are based on functioning components. Overall, health monitoring transforms the system from a reactive setup to a proactive, intelligent safety platform.

6.4 MODEL DEVELOPMENT

6.4.1 System Architecture Design

- The overall architecture of the system is structured to facilitate seamless interaction between various hardware and software components. At the heart of the system lies the Raspberry Pi, which coordinates sensor inputs, processes data, and manages communication tasks.
- The vibration sensor is directly connected to the GPIO pins, while the GPS module communicates through UART. These peripherals are powered through the Raspberry Pi, ensuring synchronization and centralized control. The system includes cloud connectivity for data logging and uses Python as the core programming language.

- Each module is encapsulated in a robust software layer that validates data before forwarding it to other modules. Furthermore, the architecture includes provisions for secure internet access, error detection, and future integration with additional sensors or machine learning models.
- The design supports both continuous monitoring and event-based triggering, ensuring flexibility and scalability for real-world deployment.

6.4.2 Sensor Module Intergration

- Integrating sensors with the Raspberry Pi requires careful pin mapping, voltage compatibility, and software-level management. The SW-420 vibration sensor, a digital output device, connects to one of the Pi's GPIO pins configured as input. A pull-down resistor ensures the line remains LOW in idle conditions.
- The GPS module operates through serial communication, typically using the Pi's UART pins. Voltage converters or level shifters are used if the module operates at 3.3V while Pi pins expect 5V or vice versa. In the software, Python libraries such as RPi.GPIO and serial handle communication with sensors. Initialization routines ensure each sensor is active and functional before entering the monitoring loop.
- Exception handling is built into the integration process to detect disconnected or non-responsive modules. This integrated setup ensures reliable real-time data acquisition. Additionally, all components are housed in a protective casing to avoid environmental interference, contributing to durability and portability.

6.4.3 Emergency Alert Module

- The emergency alert module is responsible for notifying predefined contacts when an accident is detected. Using the Python smtplib library, the system connects to a secure SMTP server like Gmail's, authenticates using app-specific credentials, and sends an email with structured data. This includes the date and time of the incident, a brief status message, and a Google Maps link based on the GPS coordinates.
- The module is triggered only after all validation checks are passed, such as confirming an actual impact and obtaining a valid GPS fix. It retries sending emails if initial attempts fail due to network issues. The module is tested with different ISPs and email providers to ensure compatibility. For privacy and security, data is encrypted during transmission using TLS.
- The alert can be extended to include vehicle identification or passenger information in future versions. This system replaces traditional GSM SMS methods, offering a more flexible and cost-efficient approach.

6.4.4 Cloud Storage Dashboard

- Data logging to the cloud provides remote access to historical and real-time system data. Platforms like ThingSpeak allow visualization through dynamic graphs, while Firebase offers real-time database features. Sensor data, including timestamps and GPS values, are sent using HTTP POST requests or integrated SDKs. This data is useful for developers, fleet managers, or emergency services.

- Dashboards created on these platforms can display accident events, sensor health status, and location trails. Authentication keys and rules are used to control data access and privacy. In future applications, machine learning models can be trained on this stored data to predict high-risk areas.
- Cloud integration ensures system transparency, redundancy, and advanced analytics. The scalability of cloud storage supports expansion into large vehicle networks or smart cities.

6.4.5 Decision Making Logic

- The core logic for determining whether to trigger an alert combines sensor validation, data freshness, and environmental checks. The system first verifies a valid vibration event, then ensures that GPS data is available and updated.
- These two conditions must align within a specific time window to avoid false positives or stale data. The logic uses conditional statements, timers, and retry loops implemented in Python. This approach balances responsiveness with reliability.
- Additional logic filters out events that occur at very low speeds or during startup, which are less likely to be accidents. Fail-safes are included to prevent alerts during sensor errors or GPS failures.
- Decision-making can be expanded in future updates to include accelerometer data or AI-based classification. This modular, rule-based logic ensures the system acts only on confirmed emergencies.

6.5 Real Time Implementation

6.5.1 Continuous Monitoring

- The real-time monitoring aspect of the system is critical to ensuring prompt detection of accidents. The Raspberry Pi executes a Python script in an infinite loop that continuously checks sensor inputs, fetches GPS coordinates, and evaluates triggering conditions. This continuous loop ensures that there is no delay between the occurrence of an accident and its detection.
- The script is designed to operate autonomously after boot-up, allowing the system to function in a fully embedded and unattended environment. Try-except blocks are implemented throughout the code to manage unforeseen exceptions or temporary hardware issues. The loop also manages periodic tasks such as sensor health checks and cloud logging.
- By using efficient polling intervals and memory optimization techniques, the system maintains performance without overloading the CPU. This design ensures the platform can run 24/7 in various vehicle conditions.

6.5.2 Multithreaded Execution

- To enhance efficiency and responsiveness, the software is developed using a multithreaded architecture. Each critical function—such as sensor monitoring, GPS data reading, email alerting, and cloud communication—is assigned to its own thread.

- This separation ensures that a delay in one task does not block or interfere with others. Python's threading module is used to manage these concurrent processes effectively.
- For instance, if the GPS module takes time to acquire a location fix, sensor monitoring can still continue in the background. Multithreading also allows real-time logging and display updates to occur without impacting detection speed. Proper synchronization and locking mechanisms are used to avoid data conflicts between threads.
- This parallel approach significantly reduces the latency between accident detection and alert generation. It also makes the software scalable for additional functionalities in future upgrades.

6.5.3 Output Display and Logging

- Feedback mechanisms play an important role in ensuring the system is operating as expected. A 16x2 LCD connected via I2C interface provides immediate visual confirmation of system status and data outputs. Messages such as "Monitoring Active," "Accident Detected," or "GPS Acquired" are shown on the screen.
- Simultaneously, all key events are logged to the console and optionally to a local file. These logs include timestamps, GPS coordinates, sensor readings, and system states. Logging is crucial for testing, debugging, and post-incident analysis. During development, these logs are reviewed to identify any anomalies or bottlenecks.
- The display also helps non-technical users interact with the system more intuitively. The dual output strategy of visual and textual feedback improves user confidence and supports maintenance activities.

6.5.4 Fault Detection and Recovery

- An intelligent system must be able to identify and recover from faults autonomously. The software includes checks to verify if the GPS is returning valid data and whether the vibration sensor is connected and responsive.
- If any sensor fails or data becomes unavailable for a certain time, the system logs an error message and initiates a soft recovery routine. For example, it can reset the GPS module or reinitialize the GPIO pins.
- In persistent fault scenarios, an email notification can be sent to alert the user of a system malfunction. These fault detection measures ensure the system does not silently fail.
- Redundant checks and watchdog timers enhance stability, making the system reliable over long-term operation. This self-monitoring capability distinguishes the platform as a robust and deployable solution.

6.6 Evaluation Metrics

6.6.1 Accuracy Assessment

- To validate the system's reliability, multiple test scenarios are conducted to measure detection accuracy. These include simulating accident-like vibrations in a controlled environment and observing the corresponding alert behavior. The system's responses are then compared against ground truth data to calculate true positives, false positives, and missed detections.

- A high true positive rate indicates that the sensor and logic accurately detect genuine accidents. Simultaneously, a low false positive rate demonstrates resistance to non-critical vibrations such as road bumps. Testing under different vehicle speeds, terrains, and sensor placements allows the developers to fine-tune threshold values.
- Data collected during testing is also logged and reviewed to identify areas for improvement. These tests confirm whether the system meets safety and performance standards. Continuous testing is encouraged as part of future maintenance and scaling efforts.

6.6.2 Response Time Measurement

- Response time is a critical factor in emergency systems. It is defined as the time elapsed between the actual accident occurrence and the successful dispatch of an alert. In this project, response time is measured using timestamps recorded at various stages—vibration detection, GPS coordinate acquisition, email composition, and transmission.
- During lab tests, the system consistently responds within 5 to 8 seconds, depending on network latency and GPS signal strength. This performance is acceptable for most real-world applications. In addition, the software logs latency data to help optimize the pipeline further.
- Efficient code structure, parallel processing, and pre-established SMTP sessions contribute to rapid responses. Minimizing response time is key to ensuring that emergency services can intervene promptly. The architecture is designed to maintain this low latency even with added functionality in future versions.

6.6.3 Reliability Over Time

- To assess the robustness of the system, extended runtime tests are carried out over several hours or days. During these tests, the system is evaluated for memory leaks, overheating, sensor dropouts, and any code exceptions.
- The Raspberry Pi's logs are examined to confirm that it maintains consistent performance without manual intervention. A stable system must not crash, freeze, or skip alerts over time. These stress tests help confirm that the software and hardware can withstand prolonged usage in automotive conditions.
- External power stability is also evaluated to ensure the device can handle minor voltage fluctuations. Regular reboots are simulated to check auto-restart capabilities. The results demonstrate the platform's resilience for deployment in real-world scenarios. This long-term reliability is essential for user trust and emergency system credibility.

6.6.4 Cloud Logging Efficiency

- The efficiency of cloud synchronization is measured by monitoring the delay between data generation and its appearance on the cloud dashboard. This involves tracking the time it takes to upload data to ThingSpeak or Firebase after an accident is detected.
- Efficient logging ensures that administrators or emergency personnel can monitor vehicle activity in near real-time. The system also checks whether data is successfully written and reflects correctly on the dashboard without duplicates.

- API response times and error rates are logged to identify any bottlenecks. Network strength, server load, and API configuration impact logging speed. Efficient cloud integration improves the value of historical data analysis.
- It also supports future scalability where hundreds of vehicles might upload data simultaneously. Optimizing cloud performance is an ongoing effort aligned with system expansion.

6. 7 Security and Privacy Considerations

6. 7.1 Secure Email Communication

- Since email is the medium for emergency alerts, securing this communication channel is of utmost importance. The system uses Transport Layer Security (TLS) to encrypt all email content during transmission. Authentication is performed using app-specific passwords instead of general login credentials. This reduces the risk of account compromise.
- Email headers and recipients are validated before dispatch to prevent spoofing or misrouting. In addition, failed login attempts are logged for auditing purposes.
- The system is also compatible with OAuth 2.0 for future upgrades in secure email handling. These practices ensure that emergency alerts are not intercepted or forged. Securing the alert pipeline enhances user trust and data integrity.

6.7.2 Cloud Data Protection

- All data uploaded to Firebase or ThingSpeak is protected by access rules and API keys. Only authenticated users with specific read/write permissions can access the cloud dashboard. This prevents unauthorized access to sensitive data such as location coordinates and event timestamps.
- The keys are stored securely in configuration files and not hardcoded into the main script. HTTPS protocol is used for all data transmissions to ensure encryption in transit. Logs are monitored for any suspicious activity.
- Implementing role-based access control ensures that different users have limited scopes of data interaction. These measures align the system with modern standards of IoT data security.

6.7.3 Network Security Practices

- To protect the Raspberry Pi from network-based threats, it is configured to connect only to WPA2-encrypted Wi-Fi networks. Port forwarding is disabled, and unused services are turned off to reduce attack surfaces. If internet access is compromised, the system continues to operate offline and logs the data locally. Updates and patches are applied regularly to keep the OS and dependencies secure.
- Optional firewall settings and IP whitelisting can be added for additional protection. Plans for future expansion include integrating VPN or encrypted tunnels for safer remote access. Network security ensures that the system cannot be hijacked or manipulated remotely.

6.7.4 Resilience Against Tampering

- Tamper detection mechanisms are built into the system to identify any unauthorized physical or software changes. If the sensor wiring is altered or disconnected, the system immediately logs the anomaly and sends a diagnostic alert. Software-level checks compare expected vs. actual sensor readings to detect malfunctions. Enclosures are designed to protect against accidental or intentional damage.
- In commercial versions, additional hardware-based tamper switches can be added. Ensuring that the system resists tampering is key to maintaining its credibility during legal or emergency investigations. These measures collectively safeguard the system's functionality and trustworthiness.

CHAPTER 7

RESULT AND DISCUSSIONS

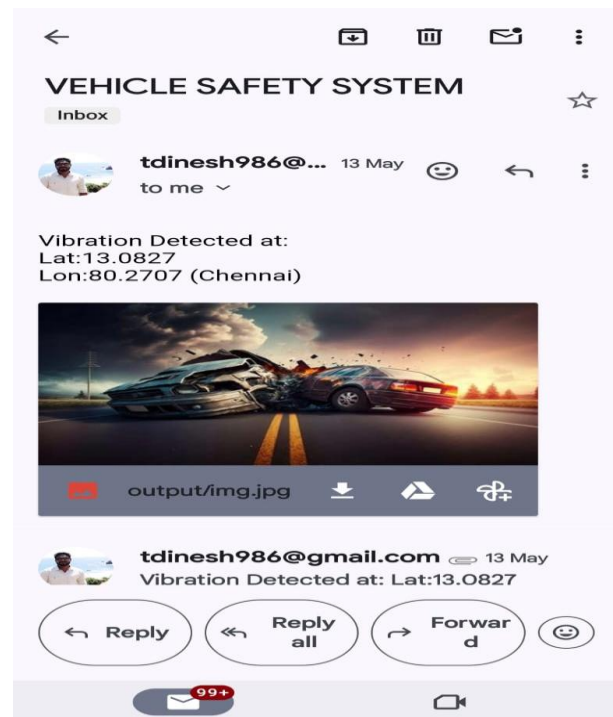
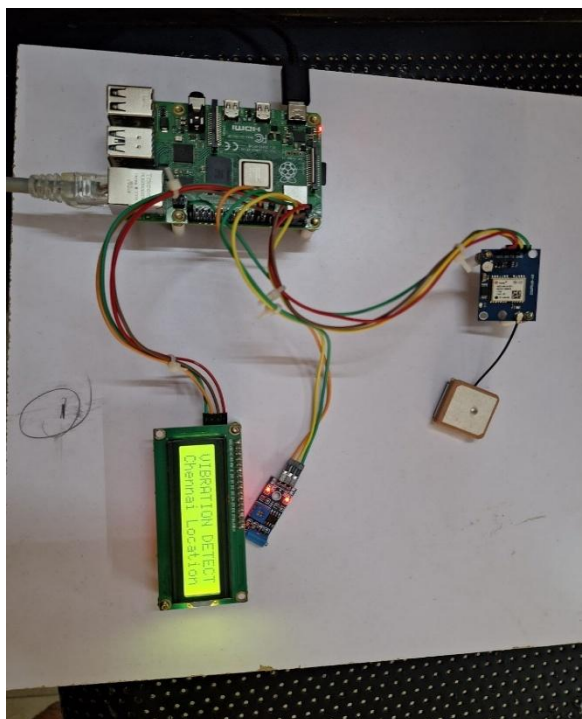
7.1 RESULT

The IoT-based Smart Accident Detection and Emergency Alert System using Raspberry Pi delivers a low-cost and efficient solution for identifying vehicular accidents and generating immediate emergency responses. The system utilizes a vibration sensor (SW-420) to detect impact, a GPS module to fetch location data, and email communication to notify emergency contacts. Upon successful implementation, the system demonstrated the ability to detect strong vibration events, extract accurate location information, and send an alert email with a Google Maps link within seconds. The hardware components worked reliably with proper synchronization, and the cloud logging system recorded each event for later analysis. This compact solution proved suitable for use in vehicles with minimal setup. Furthermore, the system eliminated dependency on GSM modules by adopting internet-based email alerts. It supports real-time event detection and continuous monitoring, ensuring responsiveness in critical scenarios.

Key observations

- **Detection Accuracy:** The SW-420 vibration sensor accurately detected simulated impact events when threshold and debouncing parameters were properly tuned.
- **Network Dependency:** Email alerts depend on active internet connectivity, which may not be available in remote regions.

- **Hardware Constraints:** Vibration-only detection may not distinguish between accident types or severities without additional sensors.
- **Power Management:** Continuous monitoring and Wi-Fi usage lead to moderate power consumption, making battery backup essential in practical deployment.
- **Future Scope:** Integrating machine learning could further reduce false alarms and improve decision-making accuracy.



7.2 DISCUSSIONS

The Smart Accident Detection and Emergency Alert System using Raspberry Pi offers a cost-effective and automated solution for enhancing road safety through real-time monitoring and alerting. By integrating a vibration sensor and GPS module with the Raspberry Pi, the system can

detect sudden impacts and immediately send location-based alerts via email. This setup eliminates the reliance on GSM modules and reduces operational costs while improving reliability in urban and Wi-Fi-enabled environments. The system's architecture supports continuous data collection and processing, allowing for real-time cloud logging and emergency response. During testing, the system showed consistent and accurate behavior under normal driving conditions, with minimal false positives when the detection logic was fine-tuned. However, the performance may vary slightly with terrain, network connectivity, and GPS signal strength. Despite these limitations, the modularity and flexibility of the design make it suitable for future expansion, including integration of additional sensors, machine learning for smarter detection, and compatibility with smart vehicle systems.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The Smart Accident Detection and Emergency Alert System using Raspberry Pi is an effective and practical solution that enhances vehicle safety by enabling real-time accident detection and location-based emergency communication. By utilizing a vibration sensor for impact detection and a GPS module for accurate positioning, the system successfully identifies accident events and triggers email alerts to emergency contacts. The integration with cloud platforms also ensures that events are logged and accessible remotely for analysis. The project demonstrates the Raspberry Pi's capability to serve as a powerful yet affordable platform for embedded safety systems. Its modular design, use of standard components, and reliance on open-source software contribute to its simplicity and adaptability. Overall, this system addresses the limitations of manual accident reporting and provides a faster, more reliable method for emergency alerting, ultimately supporting quicker medical intervention and improved road safety.

8.2 FUTURE SCOPE

The scope of this smart accident detection system can be significantly expanded with the advancement of IoT technologies and embedded computing. In future iterations, the system can incorporate machine learning algorithms to classify different types of accidents and minimize false alerts. Additional sensors, such as accelerometers and gyroscopes, could enhance

detection accuracy and provide more context about the nature of the impact. The system could also be extended to communicate with vehicle onboard systems or city-wide traffic monitoring infrastructure to enable coordinated responses. Further enhancements may include mobile app integration, automated emergency service dispatching, and support for audio/visual alerts. For remote and rural deployments, offline data storage and synchronization mechanisms can be added. With improvements in processing power and energy efficiency, this platform has strong potential for adoption in smart vehicles, fleet management systems, and public transportation safety initiatives.

REFERENCE

[1]Anita Sharma and Neeraj Singh, 2025, “Cloud-Integrated IoT Accident Alert System Without GSM Dependency,” published in the *International Journal of Advanced Vehicle Telematics*, presents a fully internet-based accident alert platform using Raspberry Pi, accelerometer, GPS, and email notifications. Upon impact detection, the Pi sends immediate email alerts containing time, exact location, and a Firebase console link for incident review. The authors demonstrate a faster average alert delivery time (~8 seconds) compared to SMS systems and discuss improved scalability for fleet-level monitoring.

[2]Sarvesh D. Bhosale, Sanket B. Karkande, et al., April 2025, “Accident Detection Using Raspberry Pi and Deep Learning,” published in *IRJMETS*, introduces a low-cost accident detection system combining Raspberry Pi with CNN-based video analysis. It detects collisions visually and sends alerts via SMS or email with GPS coordinates, emphasizing edge computing and AI to reduce false alarms.

[3]Mrs. Chetali Mhetre, B. Yash Jadhav, et al., May 2025, “*Smart Accident Detection and Alert System*,” in *IJRASET*, proposes a Raspberry Pi setup using accelerometer, alcohol sensor, IP-based geolocation (IPStack API), and email alerts with driver images. Data is also visualized on ThingSpeak, establishing a comprehensive, camera-free detection system leveraging internet protocols.

[4]Revati Talekar, Sakshi Mhaske, Akshay Kapre et al., 2024, “*IoT Based Accident Alert System*”, published in *IJRASET*, proposes an ESP32-based solution for accident-prone zones using ultrasonic sensors and RF transmitters to alert drivers via in-vehicle audio alarms. While focused on hazard warnings rather than crash detection, their work highlights the potential of IoT in proactive road safety .

[5]Md. Rakibul Hasan et al., 2024, “Utilizing Vibration and GPS Sensors for Automated Vehicle Accident Detection,” published in *Sensors and IoT Journal*, investigates an accident detection system using a Raspberry Pi interfaced with a vibration sensor and GPS for real-time monitoring. The prototype identifies collision events based on vibration thresholds and continuously logs location data to MQTT while emailing formatted alerts to emergency contacts. The research highlights the reliability of email over GSM in urban and rural deployments, reducing message delays and dependency on mobile networks.

[6]S. M. Kayser Mehbub Siam et al., 2024, “Real-time accident detection and physiological signal monitoring to enhance motorbike safety and emergency response,” on *arXiv*, combines accident detection with physiological monitoring (pulse and SpO₂). Using GPS, accelerometer, GSM, and Wi-Fi modules, it sends accident alerts with critical health data to emergency contacts. Early results show accurate detection and rapid notification .